

Step into proof for design patterns

Clean lecturer guide: what breakpoint to use, what Step Into proves, what to say, and why the browser waits when VS Code pauses the backend.

Before starting the demo

- Start FastAPI using VS Code debug mode: select FastAPI Debug and press F5.
- Open the web page first: `http://127.0.0.1:8000`.
- Disable unrelated breakpoints. Use only the breakpoints listed below.
- When VS Code stops at a backend breakpoint, the browser may keep loading. This is normal because the HTTP request is paused until you press Continue.

Important correction about line 47

When you Step Into from `app/routers/upload.py` line 29, VS Code enters `app/services/audio_validation.py` line 47. It does not stop on line 46 because line 46 is the function header. Debuggers usually stop at the first executable statement inside the function, which is line 47.

Why the web page may look like it is not loading

A breakpoint pauses the backend Python process. The browser is waiting for the FastAPI response, but FastAPI cannot respond while VS Code is paused at the breakpoint. So the browser may spin or look stuck. This is not a crash. It proves the browser request reached the backend code.

For the lecturer, say: "The page is waiting because the backend request is paused at the breakpoint. Once I press Continue, the API response returns and the page continues."

Line 47 itself is not a server startup line. It runs only when the upload validation factory is called. If the browser seems stuck after clicking upload or run separation, it is because the upload request has reached the factory and is paused.

Factory Method pattern: step-by-step

Goal: prove Client -> Creator/ConcreteCreator -> Product/ConcreteProduct during upload validation.

Step	Breakpoint / action	What to press	What it proves
1	Set breakpoint at app/routers/upload.py:29 validator = audio_validator_factory.create_validator(file.filename)	Click Upload / Run Separation after selecting WAV.	The Client is upload_audio(). It asks the factory for a validator.
2	When stopped at upload.py:29	Step Into	VS Code goes to audio_validation.py:47 . This proves the call entered the factory method implementation.
3	Now at app/services/audio_validation.py:47 suffix = Path(filename or "").suffix.lower()	Step Over or Continue to line 49.	This is WavAudioValidatorFactory, the ConcreteCreator.
4	At app/services/audio_validation.py:49 return WavAudioValidator()	Step Over / Continue.	The ConcreteCreator creates the ConcreteProduct.
5	Set breakpoint at app/routers/upload.py:30 await validator.validate(file)	Step Into	The Client uses the Product interface. VS Code enters the real ConcreteProduct.
6	VS Code enters app/services/audio_validation.py:23 filename = Path(upload_file.filename or "").name.strip()	Point to the debugger object: validator is WavAudioValidator.	ConcreteProduct runs real WAV validation.

What to say for Factory Method

- Now I will prove the Factory Method pattern using the upload validation flow.
- The Client is not a class named UploadRouter. In FastAPI, the Client role is the upload route handler, upload_audio(), inside app/routers/upload.py.
- As you can see, upload_audio() does not create WavAudioValidator directly. It calls create_validator() on the factory object.
- When I press Step Into, execution goes to audio_validation.py line 47. This is the first executable line inside WavAudioValidatorFactory.create_validator().
- At line 49, the ConcreteCreator returns WavAudioValidator. That is the ConcreteProduct.
- Then upload_audio() calls validate() through the AudioValidator Product interface. When I Step Into, execution enters WavAudioValidator, where real WAV validation happens.
- This proves object creation is separated from the route handler, so future validators can be added with less change to the upload route.

Do not say this

Do not say UploadRouter is a Python class. Say it is a diagram label for the FastAPI upload route handler. Do not say line 47 is the abstract Creator. Line 47 is inside WavAudioValidatorFactory, the ConcreteCreator. The abstract Creator is AudioValidatorFactory, but it is a contract and does not execute during upload.

Strategy pattern: step-by-step

Goal: prove Context -> Strategy call -> ConcreteStrategy during separation.

Step	Breakpoint / action	What to press	What it proves
1	Set breakpoint at app/services/separation_service.py:138 engine = self.engine_class(algorithm=algorithm, device_name="cpu")	Click Run Separation.	The SeparationEngine Context is created with the selected algorithm object.
2	Set breakpoint at app/ml/separation_engine.py:29 return self.algorithm.separate()	Step Into	The Context calls the Strategy interface through self.algorithm.
3	VS Code enters app/ml/neossnet_strategy.py:23 if model_config_path is None:	Point to the file/class name.	The runtime ConcreteStrategy is NeoSSNetStrategy.

What to say for Strategy

- Now I will prove the Strategy pattern using the separation engine.
- At this line, SeparationService creates the SeparationEngine and passes in the selected algorithm object.
- SeparationEngine is the Context. It does not depend directly on NeoSSNet internals.
- When I Step Into self.algorithm.separate(), execution enters NeoSSNetStrategy.
- This proves the engine works through the SeparationAlgorithm abstraction, and the runtime ConcreteStrategy is NeoSSNetStrategy.
- This design allows another algorithm to be added later without rewriting the engine workflow.

Fast explanation when the browser waits

Say this if the lecturer sees the browser loading: "The browser is not frozen. It is waiting because VS Code paused the backend request at the breakpoint. This is exactly what we want in a debugging demo. When I press Continue, FastAPI finishes the request and the page updates."

Minimal demo order

1. Load the page first.
2. Select the WAV file.
3. Use Factory Method breakpoints and Step Into at upload.py:29 and upload.py:30.
4. Click Continue until upload finishes.
5. Use Strategy breakpoints and Step Into at separation_engine.py:29.
6. Press Continue so the page can finish loading the result.